Ingeniería del Software II

Actividad del Tema 3: estilos arquitectónicos que conviven

Rocío Carratalá Sáez

Grado en Ingeniería Informática; Doble Grado en Matemáticas e Ingeniería Informática
Universitat de València

Curso 2026–27

1. ¿Qué estilos conviven aquí?

Una arquitectura puede combinar varios estilos porque cada uno describe una dimensión diferente: cómo se organizan los elementos y cómo colaboran (colaboración), decisiones sobre límites de proceso y despliegue, unidad de modularidad y contratos (modularidad) o plataforma de ejecución (pataforma). En cada caso debéis clasificar los estilos candidatos en tres grupos:

Presente. Existe evidencia suficiente de sus elementos, conectores y restricciones características.

Compatible, pero no demostrado. La información no contradice el estilo, aunque faltan datos para afirmarlo.

Contradicho. Algún dato del caso incumple una propiedad necesaria del estilo.

Catálogo permitido

Utilizad únicamente los estilos y enfoques trabajados en las transparencias:

Colaboración	**Lím. proceso/despl.**	**Modularidad**	**Plataforma**
Repositorio	Monolítica	Orientada a objetos	Máquinas virtuales
Tuberías y filtros	Microservicios	Orientada a servicios	Basada en la nube
Llamada y retorno	Cliente-servidor	Basada en componentes	
Por capas			
Basada en eventos			

Objetivo del ejercicio

Para cada caso planteado a continuación, indicad qué estilo o estilos son relevantes o se podrían aplicar. Para tomar las decisiones, basaos únicamente en la información proporcionada; si en algún caso es insuficiente, anotad qué evidencia o dato sería necesario para elegir el estilo apropiado.

2. Casos arquitectónicos

2.1. Caso A. Reserva universitaria

Estudiantes y personal utilizan clientes web y móvil. Todas las peticiones llegan mediante el protocolo seguro de transferencia de hipertexto (HTTPS) a una única aplicación Java. Esta aplicación se construye y publica como un solo artefacto. En su interior, presentación, aplicación, dominio e infraestructura respetan una dirección descendente de dependencias. Los módulos de reservas, usuarios y recursos acceden a una misma base de datos relacional. Ningún módulo se despliega por separado.

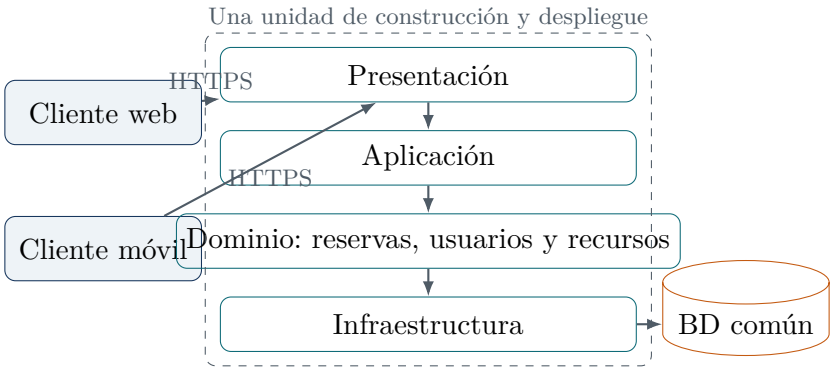


Figura 1: Arquitectura del caso A.

2.2. Caso B. Preparación de datos académicos

Cada noche se reciben ficheros de valores separados por comas (CSV, *Comma-Separated Values*). Cuatro programas ejecutables validan, anonimizan, enriquecen y agregan los registros. Cada programa recibe un flujo y produce otro con un formato acordado. Las etapas pueden sustituirse y ejecutarse en paralelo por particiones. Los resultados agregados se escriben en un almacén común consultado por informes y cuadros de mando.

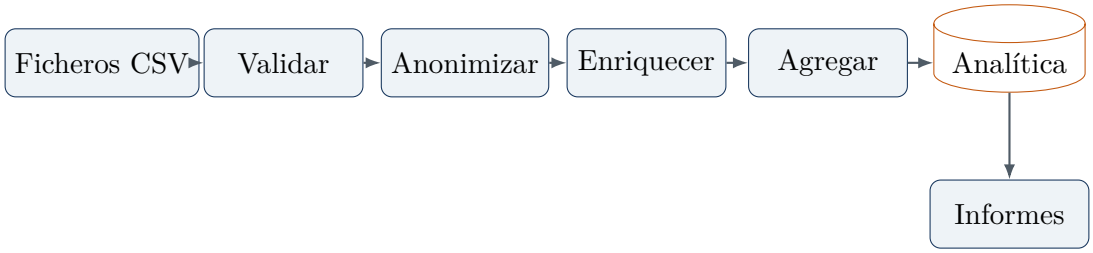


Figura 2: Arquitectura del caso B.

2.3. Caso C. Integración institucional

El portal universitario, el sistema de planificación de recursos empresariales y varias aplicaciones departamentales consumen capacidades institucionales mediante contratos versionados. Un bus de servicios empresariales (ESB, *Enterprise Service Bus*) transforma mensajes, aplica políticas y dirige las peticiones hacia servicios de matrícula, pagos e identidad. Los servicios representan capacidades amplias, comparten gobierno y se publican dentro de ventanas coordinadas. Algunos encapsulan sistemas heredados.

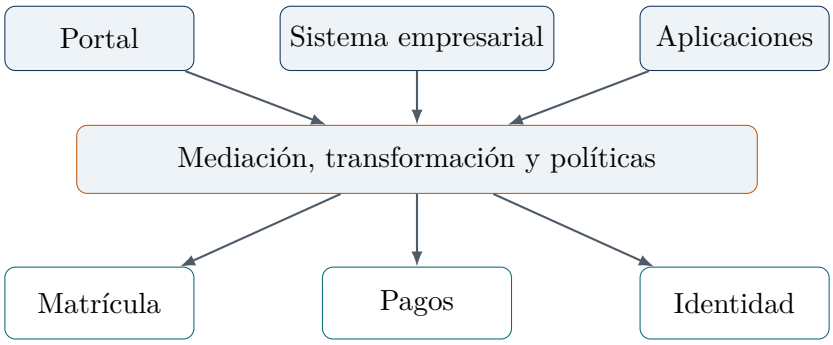


Figura 3: Arquitectura del caso C.

2.4. Caso D. Seguimiento de actividad

Tres servicios publican hechos en un intermediario de eventos gestionado por un proveedor de nube. Los servicios de notificaciones, analítica y auditoría consumen esos hechos a su ritmo. Cada servicio tiene repositorio, canal de despliegue y equipo responsable propios. Pueden publicarse y escalarse de forma independiente. La entrega de eventos puede repetirse y el orden global no está garantizado.

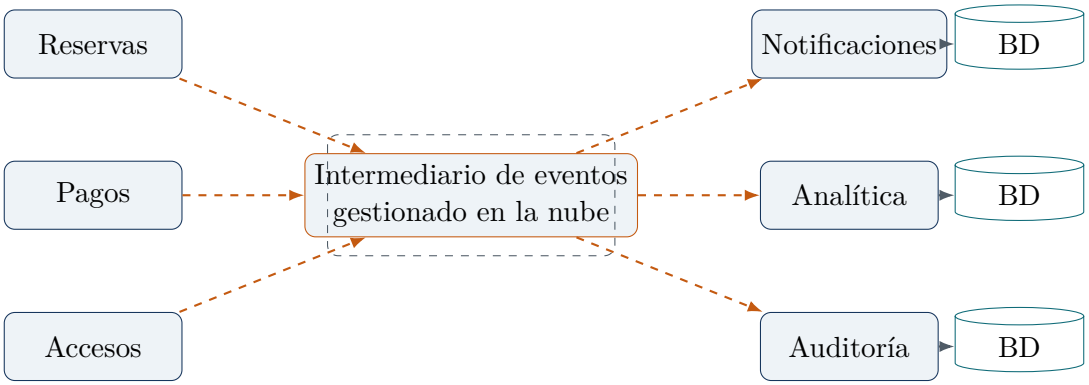


Figura 4: Arquitectura del caso D.

2.5. Caso E. Motor de flujos configurables

Las aplicaciones cliente envían una definición de flujo a un servidor. El servidor no contiene en código cada proceso: interpreta instrucciones almacenadas y mantiene el estado de ejecución. Las operaciones de envío, firma y archivado se instalan como complementos sustituibles. Cada complemento declara la interfaz que ofrece y los servicios que necesita. Definiciones y estados se conservan en una base de datos común.

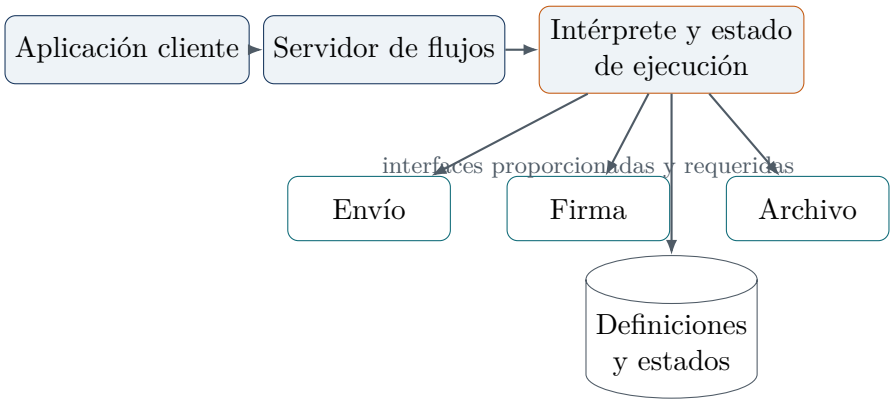


Figura 5: Arquitectura del caso E.

2.6. Caso F. Simulador de asignación

Un programa principal controla la ejecución: carga un escenario, solicita al planificador una propuesta, pide al evaluador que calcule restricciones y devuelve un informe. Todas las invocaciones son síncronas y el programa espera cada resultado antes de continuar. El modelo contiene objetos `Reserva`, `Recurso` y `PoliticaAsignacion`; cada uno conserva estado y protege sus invariantes mediante operaciones. El simulador se construye como una biblioteca ejecutable dentro de una única aplicación de escritorio.

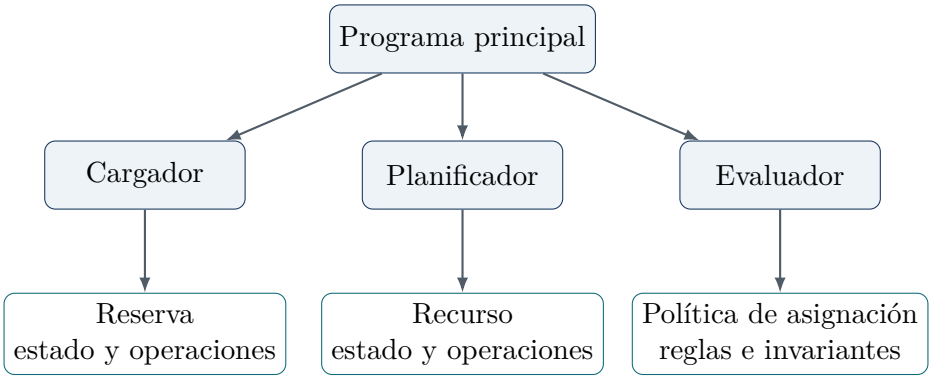


Figura 6: Arquitectura del caso F.

3. Ejercicio adicional: una modificación, una consecuencia

Aplicad al caso una de estas modificaciones. Indicad qué estilo aparece, desaparece o deja de estar demostrado y qué coste cambia.

A. Reservas y usuarios se publican ahora por separado, pero continúan escribiendo la misma base de datos.

B. Cada etapa deja de intercambiar flujos y comienza a leer y modificar directamente las mismas tablas.

C. Los servicios pueden desplegarse de forma independiente y cada uno gobierna sus propios datos; el bus se limita a transporte.

D. Todos los consumidores deben estar disponibles y responder antes de confirmar la operación del productor.

E. Los complementos se compilan dentro del servidor y no existe un contrato que permita sustituirlos.

F. El programa principal deja de dirigir la secuencia y los módulos reaccionan a eventos publicados en un canal.